

Agent Attention: On the Integration of Softmax and Linear Attention

Dongchen Han^{1*}, Tianzhu Ye^{1*}, Yizeng Han¹, Zhuofan Xia¹, Siyuan Pan², Pengfei Wan², Shiji Song¹, and Gao Huang^{1**}

¹ Department of Automation, Tsinghua University

² Kuaishou Technology

Abstract. The attention module is the key component in Transformers. While the global attention mechanism offers high expressiveness, its excessive computational cost restricts its applicability in various scenarios. In this paper, we propose a novel attention paradigm, **Agent Attention**, to strike a favorable balance between computational efficiency and representation power. Specifically, the Agent Attention, denoted as a quadruple (Q, A, K, V) , introduces an additional set of agent tokens A into the conventional attention module. The agent tokens first act as the agent for the query tokens Q to aggregate information from K and V , and then broadcast the information back to Q . Given the number of agent tokens can be designed to be much smaller than the number of query tokens, agent attention is significantly more efficient than the widely adopted Softmax attention, while preserving global context modelling capability. Interestingly, we show that the proposed agent attention is equivalent to a generalized form of linear attention. Therefore, agent attention seamlessly integrates the powerful Softmax attention and the highly efficient linear attention. Extensive experiments demonstrate the effectiveness of agent attention with various vision Transformers and across diverse vision tasks, including image classification, object detection, semantic segmentation and image generation. Notably, agent attention has shown remarkable performance in high-resolution scenarios, owing to its linear attention nature. For instance, when applied to Stable Diffusion, our agent attention accelerates generation and substantially enhances image generation quality without any additional training. Code is available at <https://github.com/LeapLabTHU/Agent-Attention>.

Keywords: Attention mechanism · Agent attention · Vision Transformer

1 Introduction

Originating from natural language processing, Transformer models have rapidly gained prominence in the field of computer vision in recent years, achieving significant success in image classification [13, 15, 18, 44], object detection [5, 46], semantic segmentation [6, 52], and multimodal tasks [33, 35, 49].

* Equal contribution.

** Corresponding Author.

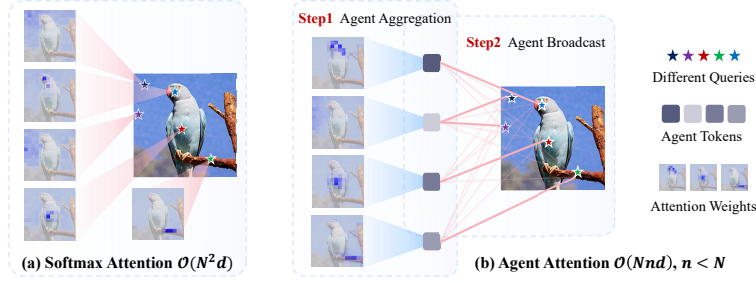


Fig. 1: An illustration of the motivation of our agent attention. (a) In Softmax attention, each query aggregates information from all features, incurring quadratic complexity. (b) Leveraging the redundancy between attention weights, agent attention uses a small number of agent tokens to act as the “agent” for queries, capturing diverse semantic information from all features, and then presenting it to each query. The attention weights are derived from DeiT-T and Agent-DeiT-T.

Nevertheless, incorporating Transformers and self-attention into the visual domain presents formidable challenges. Modern Transformer models commonly employ Softmax attention [45], which computes the similarity between each query-key pair, resulting in quadratic computation complexity with respect to the number of tokens. As a result, directly applying Softmax attention with global receptive fields to the visual tasks can lead to unmanageable computational demands. To tackle this issue, existing works [16, 17, 19, 29, 46, 50, 63] attempt to reduce computation complexity by designing efficient attention patterns. As two representatives, Swin Transformer [29] reduces the receptive field and confines self-attention calculations to local windows. PVT [46] employs a sparse attention pattern to alleviate the computational burden by reducing the number of keys and values. Despite their effectiveness, these methods inevitably compromise the capability to model long-range relationships, and are still inferior to global self-attention mechanism.

In this paper, in contrast to restricting receptive field or introducing sparsity, we propose a novel quadruplet attention paradigm (Q, A, K, V) , dubbed **Agent Attention**, which exploits redundancy between attention weights to achieve both high model expressiveness and low computation complexity. As shown in Fig. 1, in Softmax attention, each query aggregates information from all features, incurring quadratic complexity. In fact, many queries, such as those denoting sky in Fig. 1a, require similar information. Therefore, our motivation is to eliminate the direct contact between each query and key, and instead use a small number of agent tokens A to act as the “agent” for queries, capturing diverse semantic information from all features, and then presenting it to each query. As illustrated in Fig. 1b and Fig. 2c, the resulting agent attention is composed of two conventional Softmax attention operations. The first Softmax attention treats agent tokens A as *queries* to aggregate agent features V_A from all values V , and the second utilizes agent tokens A as *keys*, broadcasting the global information

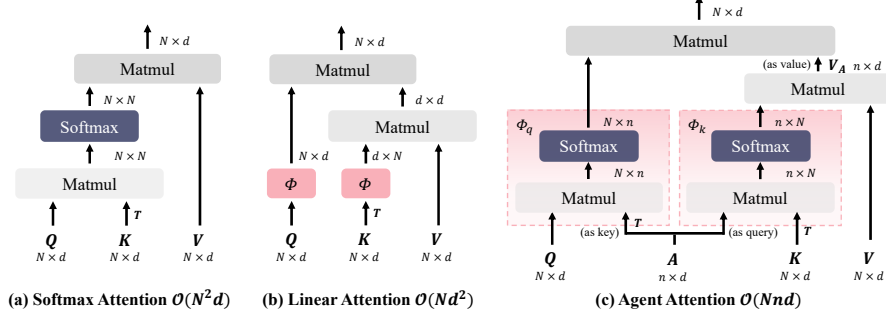


Fig. 2: Difference between Softmax attention, Linear attention and Agent attention. (a) Softmax attention computes the similarities between all query-key pairs, resulting in quadratic complexity. (b) Linear attention applies mapping function $\phi(\cdot)$ to Q and K respectively to change the computation order, reducing complexity but suffering from insufficient expressive capability. (c) Our Agent attention employs a small group of agent tokens to aggregate and broadcast global information, leading to an elegant integration of Softmax and linear attention and naturally enjoying the advantages of both high expressiveness and low computation complexity.

from agent features V_A to each query and forming the final output. Intuitively, the newly introduced tokens A serve as “agent” for the query tokens Q , as they directly collect information from K and V , and then deliver the result to Q .

Due to the intrinsic redundancy in global self-attention, the number of agent tokens can be designed to be much smaller than the number of query tokens. This property endows agent attention with high efficiency, reducing the quadratic complexity of Softmax attention to linear complexity while preserving global context modelling capability. Interestingly, as illustrated in Fig. 2, the proposed agent attention practically forms an elegant integration of Softmax and linear attention, which explains how it achieves both high efficiency and high expressiveness from a novel perspective.

We empirically verify the effectiveness of our model across diverse vision tasks, including image classification, object detection, semantic segmentation and image generation. Our method yields substantial improvements in various tasks, particularly in high-resolution scenarios. Noteworthy, our agent attention can be directly plugged into pre-trained large diffusion models, and without any additional training, it not only accelerates the generation process, but also notably improves the generation quality.

2 Related Works

Vision Transformer. Since the inception of Vision Transformer [13], self-attention has made notable strides in the realm of computer vision. However, the quadratic complexity of the prevalent Softmax attention [45] poses a challenge in applying self-attention to visual tasks. Previous works proposed various remedies

for this computational challenge. PVT [46] introduces sparse global attention, curbing computation cost by reducing the resolution of K and V . Swin Transformer [29] restricts self-attention computations to local windows and employs shifted windows to model the entire image. NAT [19] emulates convolutional operations and calculates attention within the neighborhood of each feature. DAT [50] designs a deformable attention module to achieve a data-dependent attention pattern. BiFormer [63] uses bi-level routing attention to dynamically determine areas of interest for each query. GRL [25] employs a mixture of anchored stripe attention, window attention, and channel attention to achieve efficient image restoration. However, these approaches inherently limit the global receptive field of self-attention or are vulnerable to specifically designed attention patterns, hindering their plug-and-play adaptability for general purposes.

Linear Attention. In contrast to the idea of restricting receptive fields, linear attention directly addresses the computational challenge by reducing computation complexity. The pioneer work [22] discards the Softmax function and replaces it with a mapping function ϕ applied to Q and K , thereby reducing the computation complexity to $\mathcal{O}(N)$. However, such approximations led to substantial performance degradation. To tackle this problem, Efficient Attention [41] applies the Softmax function to both Q and K . SOFT [32] and Nystromformer [53] employ matrix decomposition to further approximate Softmax operation. Castling-ViT [57] uses Softmax attention as an auxiliary training tool and fully employs linear attention during inference. FLatten Transformer [14] proposes focused function and adopts depthwise convolution to preserve feature diversity. While these methods are effective, they continue to struggle with the limited expressive power of linear attention. In the paper, rather than enhancing Softmax or linear attention, we propose agent attention which integrates these two attention types, achieving superior performance in various tasks.

3 Preliminaries

In this section, we first review the general form of self-attention in vision Transformers and briefly analyze the pros and cons of Softmax and linear attention.

3.1 General Form of Self-Attention

With an input of N tokens represented as $x \in \mathbb{R}^{N \times C}$, self-attention can be formulated as follows in each head:

$$Q = xW_Q, K = xW_K, V = xW_V, \quad O_i = \sum_{j=1}^N \frac{\text{Sim}(Q_i, K_j)}{\sum_{j=1}^N \text{Sim}(Q_i, K_j)} V_j, \quad (1)$$

where $W_{Q/K/V} \in \mathbb{R}^{C \times d}$ are projection matrices, C and d are the channel dimension of module and each head, and $\text{Sim}(\cdot, \cdot)$ denotes the similarity function.

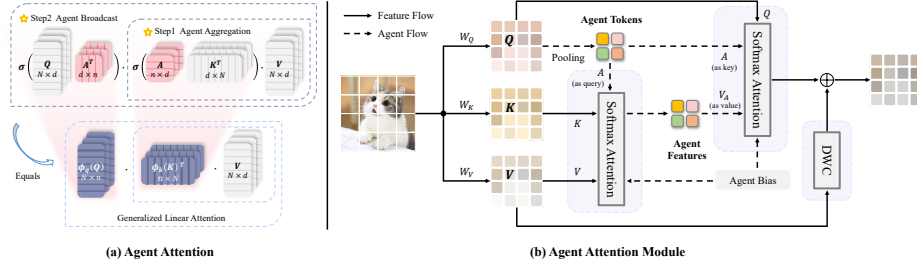


Fig. 3: An illustration of our agent attention and agent attention module. (a) Agent attention uses agent tokens to aggregate global information and distribute it to individual image tokens, resulting in a practical integration of Softmax and linear attention. $\sigma(\cdot)$ represents Softmax function. In (b), we depict the information flow of agent attention module. As a showcase, we acquire agent tokens through pooling. Subsequently, agent tokens are utilized to aggregate information from V , and Q queries features from the agent features. In addition, agent bias and DWC are adopted to add positional information and maintain feature diversity.

3.2 Softmax Attention and Linear Attention

When using $\text{Sim}(Q, K) = \exp(QK^T/\sqrt{d})$ in Eq. (1), it becomes Softmax attention [45], which has been highly successful in modern vision Transformer designs. However, Softmax attention compels to compute the similarity between all query-key pairs, resulting in $\mathcal{O}(N^2)$ complexity. Consequently, using Softmax attention with a global receptive field leads to overwhelming computation complexity. To tackle this problem, previous works attempted to reduce the number of tokens N by designing sparse global attention [46, 47] or window attention [12, 29] patterns. While effective, these strategies unavoidably compromise the self-attention’s capability for long-range modeling.

Comparably, linear attention [22] efficiently addresses the computation challenge with a linear complexity of $\mathcal{O}(N)$. Specifically, carefully designed mapping functions are applied to Q and K respectively, *i.e.*, $\text{Sim}(Q, K) = \phi(Q)\phi(K)^T$. This gives us the opportunity to change the computation order from $(\phi(Q)\phi(K)^T)V$ to $\phi(Q)(\phi(K)^TV)$ based on the associative property of matrix multiplication. As illustrated in Fig. 2, by doing so, the computation complexity with respect to token number is reduced to $\mathcal{O}(N)$. However, designing effective mapping function $\phi(\cdot)$ proves to be a nontrivial task. Simple functions [41] such as ReLU lead to significant performance drop, whereas more intricate designs [7] or matrix decomposition methods [32, 53] may introduce extra computation overhead. In general, current linear attention approaches are still inferior to Softmax attention, limiting their practical application.

4 Agent Transformer

As discussed in Sec. 3, Softmax and linear attention suffer from either excessive computation complexity or insufficient model expressiveness. Previous research

commonly treated these two attention paradigms as distinct approaches and attempted to either reduce the computation cost of Softmax attention or enhance the performance of linear attention. In this section, we propose a new attention paradigm named **Agent Attention**, which practically forms an elegant integration of Softmax and linear attention, enjoying benefits from both linear complexity and high expressiveness.

4.1 Agent Attention

To simplify, we abbreviate Softmax and linear attention as:

$$O^S = \sigma(QK^T)V \triangleq \text{Attn}^S(Q, K, V), \quad O^\phi = \phi(Q)\phi(K)^T V \triangleq \text{Attn}^\phi(Q, K, V), \quad (2)$$

where $Q, K, V \in \mathbb{R}^{N \times C}$ denote query, key and value matrices and $\sigma(\cdot)$ represents Softmax function. Then our agent attention can be written as:

$$O^A = \underbrace{\text{Attn}^S(Q, A, \underbrace{\text{Attn}^S(A, K, V)}_{\text{Agent Aggregation}})}_{\text{Agent Broadcast}}. \quad (3)$$

It is equivalent to:

$$O^A = \sigma(QA^T) \sigma(AK^T) V = \phi_q(Q)\phi_k(K)^T V = \underbrace{\text{Attn}^{\phi_q/\phi_k}(Q, K, V)}_{\text{Generalized Linear Attn}}, \quad (4)$$

where $A \in \mathbb{R}^{n \times C}$ is our newly defined agent tokens.

As shown in Eq. (3) and Fig. 3a, our agent attention consists of two Softmax attention operations, namely agent aggregation and agent broadcast. Specifically, we initially treat agent tokens A as *queries* and perform attention calculations between A , K , and V to aggregate agent features V_A from all values. Subsequently, we utilize A as *keys* and V_A as *values* in the second attention calculation with the query matrix Q , broadcasting the global information from agent features to every query token and obtaining the final output O . In this way, we avoid the computation of pairwise similarities between Q and K while preserving information exchange between each query-key pair through agent tokens.

The newly defined agent tokens A essentially serve as the *agent* for Q , aggregating global information from K and V , and subsequently broadcasting it back to Q . Practically, we set the number of agent tokens n as a small hyperparameter, achieving a linear complexity of $\mathcal{O}(Nnd)$ relative to the number of input features N while maintaining global context modeling capability. Interestingly, as shown in Eq. (4) and Fig. 3a, we practically integrate the powerful Softmax attention and efficient linear attention, establishing a generalized linear attention paradigm by employing two Softmax attention operations, with the equivalent mapping function defined as $\phi_q(Q) = \sigma(QA^T)$, $\phi_k(K) = (\sigma(AK^T))^T$.

In practice, agent tokens can be acquired through different methods, such as simply setting as a set of learnable parameters or extracting from input features

through pooling or convolution. It is worth noticing that more advanced techniques like deformed points [50] or token merging [1] can also be used to obtain agent tokens. In the default setting, we employ the simple pooling strategy to obtain agent tokens, which already works surprisingly well.

4.2 Agent Attention Module

Agent attention inherits the merits of both Softmax and linear attention. In practical use, we further make two improvements to maximize its potential.

Agent Bias. In order to better utilize positional information, we present a carefully designed *Agent Bias* for our agent attention. Specifically, inspired by RPB [40], we introduce agent bias within the attention calculation, i.e.,

$$O^A = \sigma(QA^T + B_2) \sigma(AK^T + B_1) V, \quad (5)$$

where $B_1 \in \mathbb{R}^{n \times N}$, $B_2 \in \mathbb{R}^{N \times n}$ are our agent biases. For parameter efficiency, we construct each agent bias using three bias components rather than directly setting B_1, B_2 as learnable parameters (see Appendix). Agent bias augments the vanilla agent attention with spatial information, helping different agent tokens to focus on diverse regions. As shown in Tab. 6, significant improvements can be observed upon the introduction of our agent bias terms.

Diversity Restoration Module. Although agent attention benefits from both low computation cost and high expressiveness, as generalized linear attention, it also suffers from insufficient feature diversity [14]. As a remedy, we follow [14] and adopt a depthwise convolution (DWC) module to preserve feature diversity.

Agent Attention Module. Building upon these designs, we propose a novel attention module named *Agent Attention Module*. As illustrated in Fig. 3(b), our module is composed of three parts, namely pure agent attention, agent bias and the DWC module. Our module can be formulated as:

$$O = \sigma(QA^T + B_2) \sigma(AK^T + B_1) V + \text{DWC}(V), \quad (6)$$

where $Q, K, V \in \mathbb{R}^{N \times C}$, $A \in \mathbb{R}^{n \times C}$, $B_1 \in \mathbb{R}^{n \times N}$ and $B_2 \in \mathbb{R}^{N \times n}$. In the default setting, agent tokens A is obtained through pooling, i.e., $A = \text{Pooling}(Q)$. The overall module complexity is expressed as:

$$\Omega = \underbrace{4NC^2}_{\text{Proj}} + \underbrace{NC}_{\text{Get Agents}} + \underbrace{2nNC + 2NnC}_{\text{Agent Attention}} + \underbrace{k^2NC}_{\text{DWC}}, \quad (7)$$

where N, n are the number of input features and agent tokens, and $k = 3$ is the kernel size of DWC. Notably, our model exhibits linear complexity for N .

Combining the merits of Softmax and linear attention, our module offers the following advantages:

(1) **Efficient computation and high expressive capability.** Previous work usually viewed Softmax attention and linear attention as two different attention paradigms, aiming to address their respective limitations. As a seamless

Table 1: ImageNet-1K classification results. The default input resolution is 224^2 , except $\uparrow^{384}$ denotes results on 384^2 resolution. Check the Appendix for full results.

Method	#Params	FLOPs	Top-1 Acc.
DeiT-T [44]	5.7M	1.2G	72.2
Agent-DeiT-T	6.0M	1.2G	74.9 (+2.7)
DeiT-S	22.1M	4.6G	79.8
Agent-DeiT-S	22.7M	4.4G	80.5 (+0.7)
PVT-T [46]	13.2M	1.9G	75.1
Agent-PVT-T	11.6M	2.0G	78.4 (+3.3)
PVT-S	24.5M	3.8G	79.8
Agent-PVT-S	20.6M	4.0G	82.2 (+2.4)
PVT-L	61.4M	9.8G	81.7
Agent-PVT-L	48.7M	10.4G	83.7 (+2.0)

Method	#Params	FLOPs	Top-1 Acc.
Swin-T [29]	29M	4.5G	81.3
Agent-Swin-T	29M	4.5G	82.6 (+1.3)
Swin-S	50M	8.7G	83.0
Agent-Swin-S	50M	8.7G	83.7 (+0.7)
Swin-B	88M	15.4G	83.5
Agent-Swin-B	88M	15.4G	84.0 (+0.5)
CSwin-B [12]	78M	15.0G	84.2
Agent-CSwin-B	73M	14.9G	84.7 (+0.5)
CSwin-B $\uparrow^{384}$	78M	47.0G	85.4
Agent-CSwin-B$\uparrow^{384}$	73M	46.3G	85.8 (+0.4)

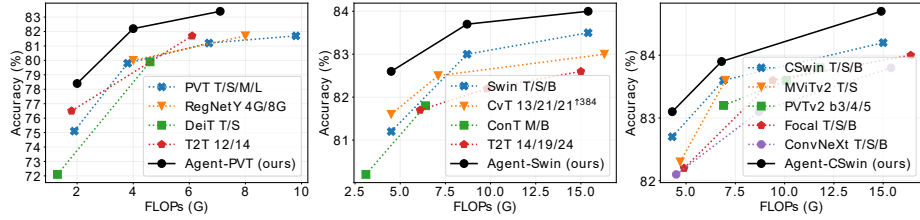


Fig. 4: Comparison with SOTA models [24, 30, 36, 44, 47, 48, 54, 56, 58] on ImageNet-1K.

integration of these two attention forms, our agent attention naturally inherits the merits of the two, enjoying both low computation complexity and high model expression ability at the same time.

(2) **Large receptive field.** Our module can adopt a large receptive field while maintaining the same amount of computation. Modern vision Transformer models typically resort to sparse attention [46, 47] or window attention [12, 29] to mitigate the computation burden of Softmax attention. Benefited from linear complexity, our model can enjoy the advantages of a large, even global receptive field while maintaining the same computation.

4.3 Implementation

Our agent attention module can serve as a plug-in module and can be easily adopted on a variety of modern vision Transformer architectures. As a showcase, we empirically apply our method to four advanced and representative Transformer models including DeiT [44], PVT [46], Swin [29] and CSwin [12]. We also apply agent attention to Stable Diffusion [37] to accelerate image generation. Detailed model architectures are shown in Appendix.

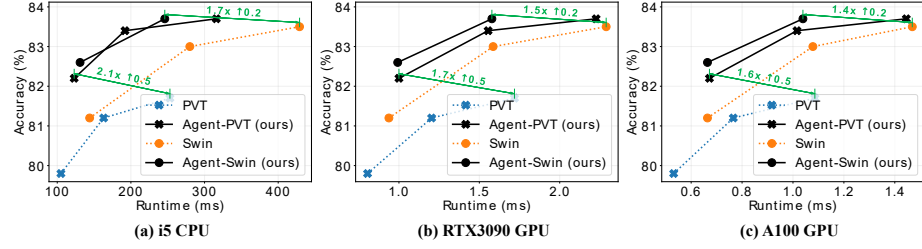
5 Experiments

To verify the effectiveness of our method, we conduct experiments on ImageNet-1K classification [9], ADE20K semantic segmentation [62], and COCO object

Table 2: Results on COCO dataset. The FLOPs are computed over backbone, FPN and detection head with an input resolution of 1280×800 . See full results in Appendix.

(a) Mask R-CNN Object Detection							
Method	FLOPs	Sch.	AP ^b	AP ^b ₅₀	AP ^b ₇₅	AP ^m	AP ^m ₅₀ AP ^m ₇₅
PVT-T	240G	1x	36.7	59.2	39.3	35.1	56.7 37.3
Agent-PVT-T	230G	1x	41.4	64.1	45.2	38.7	61.3 41.6
PVT-M	392G	1x	42.0	64.4	45.6	39.0	61.6 42.1
Agent-PVT-M	400G	1x	45.9	67.8	50.4	42.0	65.0 45.4
PVT-L	494G	1x	42.9	65.0	46.6	39.5	61.9 42.5
Agent-PVT-L	510G	1x	46.9	69.2	51.4	42.8	66.2 46.2
Swin-S	358G	1x	45.7	67.9	50.4	41.1	64.9 44.2
Agent-Swin-S	364G	1x	47.2	69.6	52.3	42.7	66.6 45.8

(b) Cascade Mask R-CNN Object Detection							
Method	FLOPs	Sch.	AP ^b	AP ^b ₅₀	AP ^b ₇₅	AP ^m	AP ^m ₅₀ AP ^m ₇₅
Swin-T	745G	1x	48.1	67.1	52.2	41.7	64.4 45.0
Agent-Swin-T	755G	1x	49.2	68.6	53.2	42.7	65.6 45.9
Swin-T	745G	3x	50.4	69.2	54.7	43.7	66.6 47.3
Agent-Swin-T	755G	3x	51.4	70.2	55.9	44.5	67.6 48.4
Swin-S	837G	3x	51.9	70.7	56.3	45.0	68.2 48.8
Agent-Swin-S	843G	3x	52.6	71.3	57.1	45.5	68.9 49.2
Swin-B	981G	3x	51.9	70.5	56.4	45.0	68.1 48.9
Agent-Swin-B	990G	3x	52.6	71.1	57.1	45.3	68.6 49.2

**Fig. 5:** Accuracy-Runtime curve on ImageNet. Runtime is tested with resolution 224^2 .

detection [27]. Additionally, we integrate agent attention into the state-of-the-art generation model, Stable Diffusion [37]. Furthermore, we construct high-resolution models with large receptive fields to maximize the benefits of agent attention. In addition, sufficient ablation experiments are conducted to show the effectiveness of each design.

5.1 ImageNet-1K Classification

ImageNet [9] comprises 1000 classes, with 1.2 million training images and 50,000 validation images. We implement our module on four representative vision Transformers and compare the top-1 accuracy on the validation split with state-of-the-art models. See Appendix for **training settings**.

Results. As depicted in Tab. 1, substituting Softmax attention with agent attention in various models results in significant performance improvements. For instance, Agent-PVT-S surpasses PVT-L while using just 30% of the parameters and 40% of the FLOPs. Additionally, we provide a comprehensive comparison with various state-of-the-art methods in Fig. 4. Our models clearly achieve a better trade-off between computation cost and model performance. These results unequivocally prove that our approach has robust advantages and is adaptable to diverse architectures.

Inference Time. We further conduct real speed measurements by deploying the models on various devices. As Fig. 5 illustrates, our models attain inference speeds 1.7 to 2.1 times faster on the CPU while simultaneously improving per-

Table 3: Results of semantic segmentation on ADE20K. The FLOPs are computed over encoders and decoders with an input image at the resolution of 512×2048 .

SemanticFPN Semantic Segmentation						UperNet Semantic Segmentation					
Backbone	Method	FLOPs	#Params	mIoU	mAcc	Backbone	Method	FLOPs	#Params	mIoU	mAcc
PVT-T	S-FPN	158G	17M	36.57	46.72	Swin-T	UperNet	945G	60M	44.51	55.61
Agent-PVT-T	S-FPN	147G	15M	40.18	51.76	Agent-Swin-T	UperNet	954G	61M	46.68	58.53
PVT-S	S-FPN	225G	28M	41.95	53.02	Swin-S	UperNet	1038G	81M	47.64	58.78
Agent-PVT-S	S-FPN	211G	24M	44.18	56.17	Agent-Swin-S	UperNet	1043G	81M	48.08	59.78
PVT-L	S-FPN	420G	65M	43.49	54.62	Swin-B	UperNet	1188G	121M	48.13	59.13
Agent-PVT-L	S-FPN	434G	52M	46.52	58.50	Agent-Swin-B	UperNet	1196G	121M	48.73	60.01

formance. On RTX3090 GPU and A100 GPU, our models also achieve 1.4x to 1.7x faster inference speeds.

5.2 Object Detection

COCO [27] object detection and instance segmentation dataset has 118K training and 5K validation images. We apply our model to RetinaNet [26], Mask R-CNN [20] and Cascade Mask R-CNN [4] frameworks to evaluate the performance of our method. A series of experiments are conducted utilizing both 1x and 3x schedules with different detection heads. As depicted in Tab. 2, our model exhibits consistent enhancements across all configurations. Agent-PVT outperforms PVT models with an increase in box AP ranging from **+3.9** to **+4.7**, while Agent-Swin surpasses Swin models by up to **+1.5** box AP. These substantial improvements can be attributed to the large receptive field brought by our design, proving the effectiveness of agent attention in high-resolution scenarios.

5.3 Semantic Segmentation

ADE20K [62] is a well-established benchmark for semantic segmentation which encompasses 20K training images and 2K validation images. We apply our model to two exemplary segmentation models, namely SemanticFPN [23] and UperNet [51]. The results are presented in Tab. 3. Remarkably, our Agent-PVT-T and Agent-Swin-T achieve **+3.61** and **+2.17** higher mIoU than their counterparts. The results show that our model is compatible with various segmentation backbones and consistently achieves improvements.

5.4 Agent Attention for Stable Diffusion

The advent of diffusion models makes it possible to generate high-resolution and high-quality images. However, current diffusion models mainly use the original Softmax attention with a global receptive field, resulting in huge computation cost and slow generation speed. In the light of this, we apply our agent attention to Stable Diffusion [37], hoping to improve the generation speed of the model. Surprisingly, after simple adjustments, the Stable Diffusion model using agent

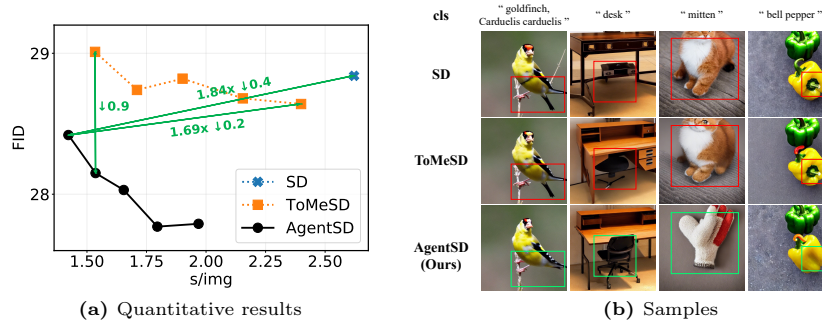


Fig. 6: (a) Quantitative Results of Stable Diffusion (SD), ToMeSD and our AgentSD. For ToMeSD, we take the merging ratios $\{0.1, 0.2, 0.3, 0.4, 0.5\}$ to construct five different models. Furthermore, we apply agent attention to each ToMeSD model to obtain the corresponding AgentSD model. (b) Samples generated by SD, ToMeSD ($r = 40\%$) and AgentSD ($r = 40\%$). The prompt is “A high quality photograph of a {cls}.”.

attention, dubbed **AgentSD**, shows a significant improvement in generation speed and produces even better image quality *without any extra training*.

Applying agent attention to Stable Diffusion. We practically apply agent attention to ToMeSD model [3]. ToMeSD reduces the number of tokens before attention calculation in Stable Diffusion, enhancing generation speed. Nonetheless, the post-merge token count remains considerable, resulting in continued complexity and latency. Hence, we replace the Softmax attention employed in ToMeSD model with our agent attention to further enhance speed. We experimentally find that when producing agent tokens through token merging [1], our agent attention can be directly applied to Stable Diffusion and ToMeSD model without any extra training. However, we are unable to apply the agent bias and DWC in this way. As a remedy, we make two simple adjustments to the agent attention, which are described in detail in Appendix. In addition, we get a significant boost by applying agent attention during early diffusion generation steps and keeping the later steps unchanged.

Quantitative Results. We follow [3] and quantitatively compare AgentSD with Stable Diffusion and ToMeSD. As displayed in Fig. 6a, ToMeSD accelerates Stable Diffusion while maintaining similar image quality. AgentSD not only further accelerates ToMeSD but also significantly enhances image generation quality. Specifically, while maintaining superior image generation quality, AgentSD achieves 1.84x and 1.69x faster generation speeds compared to Stable Diffusion and ToMeSD, respectively. At an equivalent generation speed, AgentSD produces samples with a 0.9 lower FID score compared to ToMeSD. See the experimental details and full comparison table in Appendix.

Visualization. We present some visualizations in Fig. 6b. AgentSD noticeably reduces ambiguity and generation errors in comparison to Stable Diffusion and ToMeSD. For instance, in the first column, Stable Diffusion and ToMeSD produce birds with one leg and two tails, while AgentSD’s sample does not exhibit

Table 4: Ablation on window size based on Agent-Swin-T.

	Window	FLOPs	#Param	Acc.	Diff.
Agent-Swin-T	7^2	4.5G	29M	82.0	-0.6
	14^2	4.5G	29M	82.2	-0.4
	28^2	4.5G	29M	82.4	-0.2
	56^2	4.5G	29M	82.6	Ours
Swin-T	7^2	4.5G	29M	81.3	-1.3

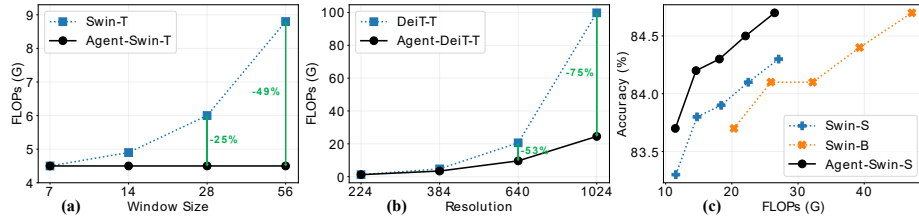


Fig. 7: (a) Comparison of FLOPs between Swin and our Agent-Swin as window size increases. (b) FLOPs comparison between DeiT and our Agent-DeiT in high-resolution scenarios. (c) Increasing resolution to $\{256^2, 288^2, 320^2, 352^2, 384^2\}$. All these models are finetuned for 30 epochs from the corresponding 224^2 resolution models.

this issue. In the third column, when provided with the prompt “A high quality photo of a mitten.”, Stable Diffusion and ToMeSD erroneously generate a cat, whereas AgentSD produces the correct image.

AgentSD for finetuning. We apply agent attention to SD-based Dreambooth [38] to verify its performance under finetuning. When finetuned, agent attention can be integrated into all diffusion steps, reaching 2.2x acceleration in generation speed compared to the original Dreambooth. Refer to Appendix for details.

5.5 Large Receptive Field and High Resolution

Large Receptive Field. Modern vision Transformers often confine self-attention calculation to local windows to reduce computation complexity, such as Swin [29]. In Tab. 4, we gradually enlarge the window size of Agent-Swin-T, ranging from 7^2 to 56^2 . Clearly, as the receptive field expands, the model’s performance consistently improves. This indicates that while the window attention pattern is effective, it inevitably compromises the long-range modeling capability of self-attention and remains inferior to global attention. As shown in Fig. 7a, unlike the quadratic complexity of Softmax attention, the linear complexity of agent attention enables us to benefit from a global receptive field while preserving identical computation complexity.

High Resolution. Limited by the quadratic complexity of Softmax attention, current vision Transformer models usually scale up by increasing model depth and width. Building on insights from [43], we discover that enhancing resolution might be a more effective approach for scaling vision Transformers, particularly

Table 5: Scaling up by increasing resolution. All these models are trained from scratch.

Method	Reso	#Params	Flops	Top-1	Method	Reso	#Params	Flops	Top-1
DeiT-B [44]	224 ²	86.6M	17.6G	81.8	PVT-L [46]	224 ²	61.4M	9.8G	81.7
DeiT-S	416 ²	22.2M	18.8G	82.9 (+1.1)	PVT-M	256 ²	44.3M	8.8G	82.2 (+0.5)
Agent-DeiT-B	224 ²	87.2M	17.6G	82.0 (+0.2)	Agent-PVT-L	224 ²	48.7M	10.4G	83.7 (+2.0)
Agent-DeiT-S	448 ²	23.1M	17.7G	83.1 (+1.3)	Agent-PVT-M	256 ²	36.1M	9.2G	83.8 (+2.1)

Table 6: Ablation on each module of agent attention.

	FLOPs	#Param	Acc.	Diff.
Vanilla Linear Attention	4.5G	29M	77.8	-4.8
Agent Attention	4.5G	29M	79.0	-3.6
+ Agent Bias	4.5G	29M	81.1	-1.5
+ DWC	4.5G	29M	82.6	Ours
Swin-T w/o PE	4.5G	29M	80.1	-2.5
+ RPE	4.5G	29M	81.3	-1.3
+ DWC	4.5G	29M	81.6	-1.0

those employing agent attention with global receptive fields. As shown in Tab. 5, Agent-DeiT-B achieves a 0.2 accuracy gain compared to DeiT-B, whereas Agent-DeiT-S at 448² resolution attains an accuracy of 83.1 with only a quarter of the parameters. We observed analogous trends when scaling the resolution of Agent-PVT-M and Agent-Swin-S (see Appendix). Fig. 7b shows the FLOPs comparison between Agent-DeiT and DeiT, with Agent-DeiT saving 75% of FLOPs for 1024² resolution images. In Fig. 7c, we progressively increase the resolution of Agent-Swin-S, Swin-S, and Swin-B. It is evident that in high-resolution scenarios, our model consistently delivers notably superior outcomes.

5.6 Ablation Study

In this section, we ablate the key components in our agent attention module to verify the effectiveness of these designs. We report the results on ImageNet-1K classification based on Agent-Swin-T.

Ablation on key designs. We substitute Softmax attention in Swin-T with vanilla linear attention, followed by a gradual introduction of agent attention, agent bias, and DWC to create Agent-Swin-T. The results are depicted in Tab. 6. Three key findings emerge: (1) Agent attention boosts accuracy by 1.2, proving its effectiveness. (2) Agent bias serves as an effective position embedding for agent attention, similar to RPE in Swin. (3) DWC is a crucial complement to unlock the capabilities of agent attention. When applying DWC to Swin-T, a modest gain of 0.3 is observed. In contrast, with DWC preserving feature diversity, agent attention delivers a much better result (+1.5).

Ablation on number of agent tokens. The model’s computation complexity can be modulated by varying the number of agent tokens. As shown in Tab. 7, shallower layers of the model have simple semantics, and judiciously decreasing

Table 7: Ablation on the number of agent tokens.

#Num of Agent Tokens				FLOPS	#Param	Acc.	Diff.
Stage1	Stage2	Stage3	Stage4				
49	49	49	49	4.7G	29M	82.6	-0.0
9	16	49	49	4.5G	29M	82.6	Ours
9	16	25	49	4.5G	29M	82.2	-0.4
4	9	49	49	4.5G	29M	82.4	-0.2
Swin-T				4.5G	29M	81.3	-1.3

Table 8: Comparison of different linear attention designs on DeiT-Tiny and Swin-Tiny.

DeiT-T Setting				Swin-T Setting			
Linear Attention	FLOPs	#Param	Acc.	Linear Attention	FLOPs	#Param	Acc.
Hydra Attn [2]	1.1G	5.7M	68.3	Hydra Attn [2]	4.5G	29M	80.7
Efficient Attn [41]	1.1G	5.7M	70.2	Efficient Attn [41]	4.5G	29M	81.0
Linear Angular Attn [57]	1.1G	5.7M	70.8	Linear Angular Attn [57]	4.5G	29M	79.4
Focused Linear Attn [14]	1.1G	6.1M	74.1	Focused Linear Attn [14]	4.5G	29M	82.1
Ours	1.2G	6.0M	74.9	Ours	4.5G	29M	82.6

the number of agent tokens in these layers does not adversely affect performance. In contrast, deeper layers have rich semantics, and reducing agent tokens in these layers leads to performance degradation. Hence, our design principle is using fewer agent tokens in the model’s shallow layers to reduce computation complexity and more agent tokens in the deep layers to better represent rich semantics. This aligns with the stripe width design principle in CSwin [12].

Comparison with Other Linear Attention. We conduct a comparison of our agent attention with other linear attention methods using DeiT-T and Swin-T. As depicted in Tab. 8, substituting the Softmax attention employed by DeiT-T and Swin-T with various linear attention methods usually results in notable performance degradation. Remarkably, our models outperform all other methods as well as the Softmax baseline.

6 Conclusion

This paper presents a new attention paradigm dubbed *Agent Attention*, which is applicable across a variety of vision Transformer models. As an elegant integration of Softmax and linear attention, agent attention enjoys both high expressive power and low computation complexity. Extensive experiments on image classification, semantic segmentation, and object detection unequivocally confirm the effectiveness of our approach, particularly in high-resolution scenarios. When integrated with Stable Diffusion, agent attention accelerates image generation and substantially enhances image quality without any extra training. Due to its linear complexity with respect to the number of tokens and its strong representation power, agent attention may pave the way for challenging tasks with super long token sequences, such as video modelling and multi-modal foundation models.

Acknowledgement

This work is supported in part by the National Key R&D Program of China under Grant 2021ZD0140407, the National Natural Science Foundation of China under Grants 42327901 and 62321005.

References

1. Bolya, D., Fu, C.Y., Dai, X., Zhang, P., Feichtenhofer, C., Hoffman, J.: Token merging: Your ViT but faster. In: ICLR (2023)
2. Bolya, D., Fu, C.Y., Dai, X., Zhang, P., Hoffman, J.: Hydra attention: Efficient attention with many heads. In: ECCVW (2022)
3. Bolya, D., Hoffman, J.: Token merging for fast stable diffusion. In: CVPRW (2023)
4. Cai, Z., Vasconcelos, N.: Cascade r-cnn: Delving into high quality object detection. In: CVPR (2018)
5. Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., Zagoruyko, S.: End-to-end object detection with transformers. In: ECCV (2020)
6. Cheng, B., Schwing, A., Kirillov, A.: Per-pixel classification is not all you need for semantic segmentation. In: NeurIPS (2021)
7. Choromanski, K., Likhoshesterov, V., Dohan, D., Song, X., Gane, A., Sarlos, T., Hawkins, P., Davis, J., Mohiuddin, A., Kaiser, L., et al.: Rethinking attention with performers. In: ICLR (2021)
8. Cubuk, E.D., Zoph, B., Shlens, J., Le, Q.V.: Randaugment: Practical automated data augmentation with a reduced search space. In: CVPRW (2020)
9. Deng, J., Dong, W., Socher, R., Li, L.J., Li, K., Fei-Fei, L.: Imagenet: A large-scale hierarchical image database. In: CVPR (2009)
10. Dettmers, T., Lewis, M., Shleifer, S., Zettlemoyer, L.: 8-bit optimizers via block-wise quantization. In: ICLR (2022)
11. Dhariwal, P., Nichol, A.: Diffusion models beat gans on image synthesis. In: NeurIPS (2021)
12. Dong, X., Bao, J., Chen, D., Zhang, W., Yu, N., Yuan, L., Chen, D., Guo, B.: Cswin transformer: A general vision transformer backbone with cross-shaped windows. In: CVPR (2022)
13. Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., et al.: An image is worth 16x16 words: Transformers for image recognition at scale. In: ICLR (2021)
14. Han, D., Pan, X., Han, Y., Song, S., Huang, G.: Flatten transformer: Vision transformer using focused linear attention. In: ICCV (2023)
15. Han, Y., Han, D., Liu, Z., Wang, Y., Pan, X., Pu, Y., Deng, C., Feng, J., Song, S., Huang, G.: Dynamic perceiver for efficient visual recognition. In: ICCV (2023)
16. Han, Y., Huang, G., Song, S., Yang, L., Wang, H., Wang, Y.: Dynamic neural networks: A survey. TPAMI (2021)
17. Han, Y., Liu, Z., Yuan, Z., Pu, Y., Wang, C., Song, S., Huang, G.: Latency-aware unified dynamic networks for efficient image recognition. TPAMI (2024)
18. Han, Y., Pu, Y., Lai, Z., Wang, C., Song, S., Cao, J., Huang, W., Deng, C., Huang, G.: Learning to weight samples for dynamic early-exiting networks. In: ECCV (2022)
19. Hassani, A., Walton, S., Li, J., Li, S., Shi, H.: Neighborhood attention transformer. In: CVPR (2023)

20. He, K., Gkioxari, G., Dollár, P., Girshick, R.: Mask r-cnn. In: ICCV (2017)
21. Heusel, M., Ramsauer, H., Unterthiner, T., Nessler, B., Hochreiter, S.: Gans trained by a two time-scale update rule converge to a local nash equilibrium. In: NeurIPS (2017)
22. Katharopoulos, A., Vyas, A., Pappas, N., Fleuret, F.: Transformers are rnns: Fast autoregressive transformers with linear attention. In: ICML (2020)
23. Kirillov, A., Girshick, R., He, K., Dollár, P.: Panoptic feature pyramid networks. In: CVPR (2019)
24. Li, Y., Wu, C.Y., Fan, H., Mangalam, K., Xiong, B., Malik, J., Feichtenhofer, C.: Mvitv2: Improved multiscale vision transformers for classification and detection. In: CVPR (2022)
25. Li, Y., Fan, Y., Xiang, X., Demandolx, D., Ranjan, R., Timofte, R., Van Gool, L.: Efficient and explicit modelling of image hierarchies for image restoration. In: CVPR (2023)
26. Lin, T.Y., Goyal, P., Girshick, R., He, K., Dollár, P.: Focal loss for dense object detection. In: ICCV (2017)
27. Lin, T.Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., Zitnick, C.L.: Microsoft coco: Common objects in context. In: ECCV (2014)
28. Liu, L., Ren, Y., Lin, Z., Zhao, Z.: Pseudo numerical methods for diffusion models on manifolds. In: ICLR (2022)
29. Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., Guo, B.: Swin transformer: Hierarchical vision transformer using shifted windows. In: ICCV (2021)
30. Liu, Z., Mao, H., Wu, C.Y., Feichtenhofer, C., Darrell, T., Xie, S.: A convnet for the 2020s. In: CVPR (2022)
31. Loshchilov, I., Hutter, F.: Decoupled weight decay regularization. In: ICLR (2018)
32. Lu, J., Yao, J., Zhang, J., Zhu, X., Xu, H., Gao, W., Xu, C., Xiang, T., Zhang, L.: Soft: Softmax-free transformer with linear complexity. In: NeurIPS (2021)
33. Pan, X., Ye, T., Han, D., Song, S., Huang, G.: Contrastive language-image pre-training with knowledge graphs. In: NeurIPS (2022)
34. Polyak, B.T., Juditsky, A.B.: Acceleration of stochastic approximation by averaging. *SIAM Journal on Control and Optimization* (1992)
35. Radford, A., Kim, J.W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., Sutskever, I.: Learning transferable visual models from natural language supervision. In: ICML (2021)
36. Radosavovic, I., Kosaraju, R.P., Girshick, R., He, K., Dollár, P.: Designing network design spaces. In: CVPR (2020)
37. Rombach, R., Blattmann, A., Lorenz, D., Esser, P., Ommer, B.: High-resolution image synthesis with latent diffusion models. In: CVPR (2022)
38. Ruiz, N., Li, Y., Jampani, V., Pritch, Y., Rubinstein, M., Aberman, K.: Dream-booth: Fine tuning text-to-image diffusion models for subject-driven generation. In: CVPR (2023)
39. Seitzer, M.: pytorch-fid: FID Score for PyTorch. <https://github.com/mseitzer/pytorch-fid> (August 2020), version 0.3.0
40. Shaw, P., Uszkoreit, J., Vaswani, A.: Self-attention with relative position representations. In: ACL (2018)
41. Shen, Z., Zhang, M., Zhao, H., Yi, S., Li, H.: Efficient attention: Attention with linear complexities. In: WACV (2021)
42. Song, J., Meng, C., Ermon, S.: Denoising diffusion implicit models. In: ICLR (2021)
43. Tan, M., Le, Q.: Efficientnet: Rethinking model scaling for convolutional neural networks. In: ICML (2019)

44. Touvron, H., Cord, M., Douze, M., Massa, F., Sablayrolles, A., Jégou, H.: Training data-efficient image transformers & distillation through attention. In: ICML (2021)
45. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. In: NeurIPS (2017)
46. Wang, W., Xie, E., Li, X., Fan, D.P., Song, K., Liang, D., Lu, T., Luo, P., Shao, L.: Pyramid vision transformer: A versatile backbone for dense prediction without convolutions. In: ICCV (2021)
47. Wang, W., Xie, E., Li, X., Fan, D.P., Song, K., Liang, D., Lu, T., Luo, P., Shao, L.: Pvt v2: Improved baselines with pyramid vision transformer. Computational Visual Media (2022)
48. Wu, H., Xiao, B., Codella, N., Liu, M., Dai, X., Yuan, L., Zhang, L.: Cvt: Introducing convolutions to vision transformers. In: ICCV (2021)
49. Xia, Z., Han, D., Han, Y., Pan, X., Song, S., Huang, G.: Gsva: Generalized segmentation via multimodal large language models. In: CVPR (2024)
50. Xia, Z., Pan, X., Song, S., Li, L.E., Huang, G.: Vision transformer with deformable attention. In: CVPR (2022)
51. Xiao, T., Liu, Y., Zhou, B., Jiang, Y., Sun, J.: Unified perceptual parsing for scene understanding. In: ECCV (2018)
52. Xie, E., Wang, W., Yu, Z., Anandkumar, A., Alvarez, J.M., Luo, P.: Segformer: Simple and efficient design for semantic segmentation with transformers. In: NeurIPS (2021)
53. Xiong, Y., Zeng, Z., Chakraborty, R., Tan, M., Fung, G., Li, Y., Singh, V.: Nyströmformer: A nyström-based algorithm for approximating self-attention. In: AAAI (2021)
54. Yan, H., Li, Z., Li, W., Wang, C., Wu, M., Zhang, C.: Contnet: Why not use convolution and transformer at the same time? arXiv preprint arXiv:2104.13497 (2021)
55. Yang, C., Xu, J., De Mello, S., Crowley, E.J., Wang, X.: Gpvit: a high resolution non-hierarchical vision transformer with group propagation. In: ICLR (2023)
56. Yang, J., Li, C., Zhang, P., Dai, X., Xiao, B., Yuan, L., Gao, J.: Focal self-attention for local-global interactions in vision transformers. In: NeurIPS (2021)
57. You, H., Xiong, Y., Dai, X., Wu, B., Zhang, P., Fan, H., Vajda, P., Lin, Y.C.: Castling-vit: Compressing self-attention via switching towards linear-angular attention at vision transformer inference. In: CVPR (2023)
58. Yuan, L., Chen, Y., Wang, T., Yu, W., Shi, Y., Jiang, Z.H., Tay, F.E., Feng, J., Yan, S.: Tokens-to-token vit: Training vision transformers from scratch on imagenet. In: ICCV (2021)
59. Yun, S., Han, D., Oh, S.J., Chun, S., Choe, J., Yoo, Y.: Cutmix: Regularization strategy to train strong classifiers with localizable features. In: ICCV (2019)
60. Zhang, H., Cisse, M., Dauphin, Y.N., Lopez-Paz, D.: mixup: Beyond empirical risk minimization. In: ICLR (2018)
61. Zhong, Z., Zheng, L., Kang, G., Li, S., Yang, Y.: Random erasing data augmentation. In: AAAI (2020)
62. Zhou, B., Zhao, H., Puig, X., Xiao, T., Fidler, S., Barriuso, A., Torralba, A.: Semantic understanding of scenes through the ade20k dataset. IJCV (2019)
63. Zhu, L., Wang, X., Ke, Z., Zhang, W., Lau, R.W.: Biformer: Vision transformer with bi-level routing attention. In: CVPR (2023)

Appendix

A. Comparison with Related Works

Agent attention shares some similarities with two related works, namely GPViT [55] and GRL [25]. In this section, we provide a detailed analysis on their fundamental distinctions and the superiority of our work.

Firstly, agent attention’s design is novel and superior. Agent attention introduces a set of agent tokens A to act as the “agent” for all queries Q , where A is usually directly acquired from the query space, i.e. $A = f(Q)$. In contrast, GRL [25] uses anchors A projected from the input X , i.e. $A = \text{Proj}(X)$, to compute cross-scale similarity (see its Fig. 6). GPViT [55] uses learnable tokens and additional MLPs to achieve global modeling (see its Fig. 3). As a result, agent attention can be applied to existing models in a training-free manner, *e.g.*, our AgentSD, which is impractical for GRL [25] and GPViT [55] since they need extra training of the projections or MLPs.

Secondly, our perspective of generalized linear attention is unique and essential. This perspective enables us to unleash the potential of agent attention with lightweight linear attention enhancements like DWC. In contrast, without such a view, GPViT [55] and GRL [25] have to compromise with other techniques, such as heavy MLP, window attention, or channel attention, to achieve comparable results.

Thirdly, agent attention is a superior alternative to Softmax attention, acting as a versatile module for general purposes, while GPViT [55] and GRL [25] are not plug-in modules and are limited to specific tasks.

B. Composition of Agent Bias

As mentioned in the main paper, to better utilize positional information, we present a carefully designed *Agent Bias* for our agent attention, i.e.,

$$O^A = \sigma(QA^T + B_2) \sigma(AK^T + B_1) V, \quad (8)$$

where $B_1 \in \mathbb{R}^{n \times N}$, $B_2 \in \mathbb{R}^{N \times n}$ are our agent biases. For parameter efficiency, we construct each agent bias using three bias components rather than directly setting B_1, B_2 as learnable parameters. For instance, values in B_1 are derived from column bias $B_{1c} \in \mathbb{R}^{n \times 1 \times w}$, row bias $B_{1r} \in \mathbb{R}^{n \times h \times 1}$ and block bias $B_{1b} \in \mathbb{R}^{n \times h_0 \times w_0}$, where h, w are the height and width of feature map or attention window, while h_0, w_0 are predefined hyperparameters much smaller than h, w . During the computation of attention weights, B_{1c}, B_{1r}, B_{1b} are resized to $B'_{1c}, B'_{1r}, B'_{1b} \in \mathbb{R}^{n \times h \times w}$ by repeating or interpolating, and $B_1 = (B'_{1c} + B'_{1r} + B'_{1b}).\text{reshape}(n, N)$ is used as the full agent bias.

C. Agent Attention for Stable Diffusion

C.1. Adjustments

As discussed in the main paper, when producing agent tokens through token merging, our agent attention can be directly applied to the Stable Diffusion model without any extra training. However, we are unable to apply the agent bias and DWC without training. As a remedy, we make two simple adjustments to our agent attention. Moreover, we change our agent attention module from

$$O = \sigma(QA^T + B_2) \sigma(AK^T + B_1) V + \text{DWC}(V), \quad (9)$$

to

$$O = \sigma(QA^T) \sigma(AK^T) V + kV, \quad (10)$$

where k is a predefined hyperparameter. On the other hand, compared to the original softmax attention, the two softmax attention operations of agent attention may result in smoother feature distribution without training. In the light of this, we slightly increase the scale used for the second Softmax attention, i.e., agent broadcast.

C.2. Experiment Details

To quantitatively compare AgentSD with Stable Diffusion and ToMeSD, we follow [3] and employ Stable Diffusion v1.5 to generate 2,000 512^2 images of ImageNet-1k [9] classes, featuring two images per class, using 50 PLMS [28] diffusion steps with a cfg scale [11] of 7.5. Subsequently, we calculate FID [21] scores between these 2,000 samples and 50,000 ImageNet-1k validation examples, employing [39]. To assess speed, we calculate the average generation time of all 2,000 samples on a single RTX4090 GPU.

Complete quantitative results are presented in Tab. 9. Compared to SD and ToMeSD, our AgentSD not only accelerates generation and reduces memory usage, but also significantly improves image generation quality.

C.3. Ablation

We further ablate the adjustments we made when applying agent attention to Stable Diffusion. As evident in Tab. 10 and Tab. 11, both adjustments to agent attention enhance the quality of AgentSD generation. Tab. 12 demonstrates that applying agent attention in the early stages yields substantial performance enhancements.

Table 9: Quantitative Results of Stable Diffusion, ToMeSD and our AgentSD. GB/img is measured as the total memory usage change when increasing batch size by 1.

Method	r	FID	s/img	GB/img
SD [37]	0	28.84	2.62	3.13
ToMeSD [3]	0.1	28.64	2.40	2.55
	0.2	28.68	2.15	2.03
	0.3	28.82	1.90	2.09
	0.4	28.74	1.71	1.69
	0.5	29.01	1.53	1.47
AgentSD	0.1	27.79	1.97	1.77
	0.2	27.77	1.80	1.60
	0.3	28.03	1.65	2.05
	0.4	28.15	1.54	1.55
	0.5	28.42	1.42	1.21

Table 10: Ablation on factor k of Eq. (10).

k	0	0.025	0.075	0.15
FID	28.80	28.67	28.42	28.61

C.4. AgentSD for finetuning

Our agent attention module is also applicable in finetuning scenarios. To verify this, we select subject-driven task as an example and apply agent attention to SD-based Dreambooth [38]. We experimentally find that finetuning enables the integration of the agent attention module into all diffusion generation steps, reaching **2.2x** acceleration in generation speed compared to the original Dreambooth without sacrificing image quality. Additionally, time and memory cost during finetuning can be reduced as well.

Task and baseline. The diffusion subject-driven generation task entails maintaining the appearance of a given subject while generating novel renditions of it in different contexts, e.g., generating a photo of your pet dog dancing. Dreambooth [38] effectively addresses this task by finetuning a pretrained text-to-image diffusion model, binding a unique identifier with the given subject. Novel images of the subject can then be synthesized with the unique identifier.

Applying agent attention to Dreambooth. As previously discussed, Dreambooth [38] involves an additional finetuning process. We explore two approaches to applying agent attention to Dreambooth: (1) applying it *only during generation* and (2) applying it during *both finetuning and generation*. The first method is the same as the AgentSD detailed in the main paper, where we commonly apply agent attention to the early 40% of generation steps, achieving around a 1.7x speedup (merging ratio $r = 0.4$). However, applying agent attention to more diffusion steps for further acceleration leads to a decrease in image details and quality, as shown in Tab. 12 and the penultimate line of Fig. 8. Conversely, adopting the second approach, where the agent attention module is applied to all

Table 11: Ablation on scale used for the second Softmax attention.

Scale	$d^{-0.5}$	$d^{-0.25}$	$d^{-0.15}$	$d^{-0.05}$
FID	28.86	28.64	28.42	28.60

Table 12: Ablation on how many diffusion steps to apply agent attention.

Steps	early 20%	early 40%	early 60%	early 80%
FID	28.58	28.42	28.83	29.77
s/img	1.50	1.42	1.39	1.34

steps in both finetuning and generation, results in a 2.2x speedup in generation without sacrificing performance. Additionally, both time and memory costs in finetuning are reduced by around 15%, enabling model finetuning with less than 12GB of GPU memory in approximately 7 minutes on a single RTX4090 GPU. The last row in Fig. 8 shows the results of this setting.

Dataset and experiment details. We adopt the dataset provided by Dreambooth [38], which comprises 30 subjects of 15 different classes. It features live subjects and objects captured in various conditions, environments, and angles. We employ pretrained Stable Diffusion v1.5 and apply agent attention to all diffusion generation steps. The merging ratio r is set to 0.4, k is set to 0.075 and the scale for the second softmax attention is set to $d^{-0.15}$. We finetune all models for 800 iterations with a learning rate of 1e-6, utilizing 8-bit AdamW [10] as the optimizer. We follow [38] and select *sks* as the unique identifier for all settings. Novel synthesized images are sampled using the DDIM [42] sampler with 100 generation steps on a single RTX4090 GPU.

Visualization and discussion. Synthesized subject-driven images are shown in Fig. 8. We make two key observations: (1) Dreambooth with agent attention applied during finetuning and generation equals or surpasses the baseline Dreambooth in terms of fidelity and editability, and (2) employing agent attention during finetuning further enhances the fidelity and detail quality of synthesized images, enabling us to apply agent attention to all diffusion steps for more speedup. For the first observation, the first column shows that our method ensures the synthesized dog’s color aligns more consistently with input images compared to the original Dreambooth and maintains comparable editability. For the second observation, comparing the last two rows of the third column reveals that applying agent attention to all diffusion steps without finetuning yields a blurry image, whereas our method produces a clearer and sharper depiction of the duck toy. Additionally, in the fifth column, our method accurately generates the cat’s eyes, whereas agent attention without finetuning fails in this aspect.

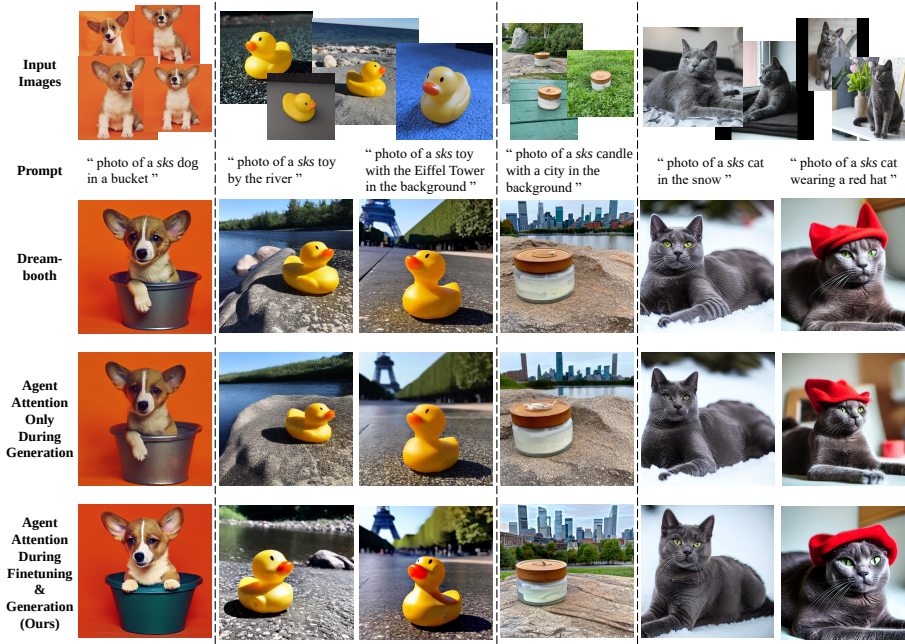


Fig. 8: Samples generated by Dreambooth and our Agent Dreambooth with the same seed. In the second-to-last line, we apply agent attention to all diffusion steps *only during generation*, leading to a slight decline in image quality as expected. In the last row, agent attention is incorporated into all steps in *both finetuning and generation*, resulting in a 2.2x speedup without compromising image quality. **Zoom in for best view.**

D. Dataset and Training Setup

D.1. ImageNet

Training settings. To ensure a fair comparison, we train our agent attention model with the same settings as the corresponding baseline model. Specifically, we employ AdamW [31] optimizer to train all our models from scratch for 300 epochs, using a cosine learning rate decay and 20 epochs of linear warm-up. We set the initial learning rate to 1×10^{-3} for a batch size of 1024 and linearly scale it *w.r.t.* the batch size. Following DeiT [44], we use RandAugment [8], Mixup [60], CutMix [59], and random erasing [61] to prevent overfitting. We also apply a weight decay of 0.05. To align with [12], we incorporate EMA [34] into the training of our Agent-CSwin models. For finetuning at larger resolutions, we follow the settings in [12, 29] and finetune the models for 30 epochs.

D.2. COCO

Training settings. COCO [27] object detection and instance segmentation dataset has 118K training and 5K validation images. We use a subset of 80k

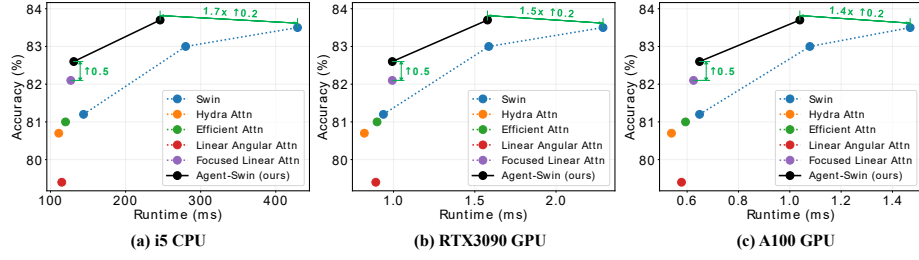


Fig. 9: Runtime comparison with other linear attention methods.

samples as training set and 35k for validation. Backbones are pretrained on ImageNet dataset with AdamW, following the training configurations mentioned in the original paper. Standard data augmentations including resize, random flip and normalize are applied. We set learning rate to $1e-4$ and follow the 1x learning schedule: the whole network is trained for 12 epochs and the learning rate is divided by 10 at the 8th and 11th epoch respectively. For some models, we utilize 3x schedule: the network is trained for 36 epochs and the learning rate is divided by 10 at the 27th and 33rd epoch. All mAP results in the main paper are tested with input image size (3, 1333, 800).

Numbers of agent tokens. We use the ImageNet pretrained model as the backbone, which is trained with numbers of agent tokens n set to [9, 16, 49, 49] for the four stages respectively. As dense prediction tasks involve higher-resolution images than ImageNet, we appropriately increase the numbers of agent tokens to better preserve the rich information. Specifically, for all the Agent-PVT models, we assign the numbers of agent tokens for the four stages as [144, 256, 784, 784], while for all Agent-Swin models, we allocate [81, 144, 196, 49]. We employ bilinear interpolation to adapt the agent bias to the increased numbers of agent tokens n . The same strategy is applied to ADE20k experiments as well.

D.3. ADE20K

Training settings. ADE20K [62] is a well-established benchmark for semantic segmentation which encompasses 20K training images and 2K validation images. Backbones are pretrained on ImageNet with AdamW, following the training configurations mentioned in the original paper. For UperNet [51], we use AdamW to optimize, and set the initial learning rate as $6e-5$ with a linear warmup of 1,500 iterations. Models are trained for 160K iterations in total. For Semantic FPN [23], we optimize the models using AdamW for 40k iterations with an initial learning rate of $2e-4$. We randomly resize and crop the image to 512×512 for training, and re-scale to have a shorter side of 512 pixels during testing.

E. Complete Experimental Results

Full classification results. We provide the full ImageNet-1K classification results (including high-resolution results) in Tab. 13. It is obvious that substituting

Table 13: Comparisons of agent attention with other vision transformer backbones on the ImageNet-1K classification task.

Method	Reso	#Params	Flops	Top-1
DeiT-T [44]	224 ²	5.7M	1.2G	72.2
Agent-DeiT-T	224 ²	6.0M	1.2G	74.9 (+2.7)
DeiT-S	224 ²	22.1M	4.6G	79.8
Agent-DeiT-S	224 ²	22.7M	4.4G	80.5 (+0.7)
DeiT-B	224 ²	86.6M	17.6G	81.8
Agent-DeiT-B	224 ²	87.2M	17.6G	82.0 (+0.2)
Agent-DeiT-S	448 ²	23.1M	17.7G	83.1 (+1.3)
PVT-T [46]	224 ²	13.2M	1.9G	75.1
Agent-PVT-T	224 ²	11.6M	2.0G	78.4 (+3.3)
PVT-S	224 ²	24.5M	3.8G	79.8
Agent-PVT-S	224 ²	20.6M	4.0G	82.2 (+2.4)
PVT-M	224 ²	44.2M	6.7G	81.2
Agent-PVT-M	224 ²	35.9M	7.0G	83.4 (+2.2)
PVT-L	224 ²	61.4M	9.8G	81.7
Agent-PVT-L	224 ²	48.7M	10.4G	83.7 (+2.0)
Agent-PVT-M	256 ²	36.1M	9.2G	83.8 (+2.1)
Swin-T [29]	224 ²	29M	4.5G	81.3
Agent-Swin-T	224 ²	29M	4.5G	82.6 (+1.3)
Swin-S	224 ²	50M	8.7G	83.0
Agent-Swin-S	224 ²	50M	8.7G	83.7 (+0.7)
Swin-B	224 ²	88M	15.4G	83.5
Agent-Swin-B	224 ²	88M	15.4G	84.0 (+0.5)
Agent-Swin-S	288 ²	50M	14.6G	84.1 (+0.6)
Swin-B	384 ²	88M	47.0G	84.5
Agent-Swin-B	384 ²	88M	46.3G	84.9 (+0.4)
CSwin-T [12]	224 ²	23M	4.3G	82.7
Agent-CSwin-T	224 ²	21M	4.3G	83.1 (+0.4)
CSwin-S	224 ²	35M	6.9G	83.6
Agent-CSwin-S	224 ²	33M	6.8G	83.9 (+0.3)
CSwin-B	224 ²	78M	15.0G	84.2
Agent-CSwin-B	224 ²	73M	14.9G	84.7 (+0.5)
CSwin-B	384 ²	78M	47.0G	85.4
Agent-CSwin-B	384 ²	73M	46.3G	85.8 (+0.4)

Table 14: Results on COCO dataset. The FLOPs are computed over backbone, FPN and detection head with input resolution of 1280×800.

(a) Mask R-CNN Object Detection on COCO								
Method	FLOPs	Sch.	AP ^b	AP ₅₀ ^b	AP ₇₅ ^b	AP ^m	AP ₅₀ ^m	AP ₇₅ ^m
PVT-T	240G	1x	36.7	59.2	39.3	35.1	56.7	37.3
Agent-PVT-T	230G	1x	41.4	64.1	45.2	38.7	61.3	41.6
PVT-S	305G	1x	40.4	62.9	43.8	37.8	60.1	40.3
Agent-PVT-S	293G	1x	44.5	67.0	49.1	41.2	64.4	44.5
PVT-M	392G	1x	42.0	64.4	45.6	39.0	61.6	42.1
Agent-PVT-M	400G	1x	45.9	67.8	50.4	42.0	65.0	45.4
PVT-L	494G	1x	42.9	65.0	46.6	39.5	61.9	42.5
Agent-PVT-L	510G	1x	46.9	69.2	51.4	42.8	66.2	46.2
Swin-T	267G	1x	43.7	66.6	47.7	39.8	63.3	42.7
Agent-Swin-T	276G	1x	44.6	67.5	48.7	40.7	64.4	43.4
Swin-T	267G	3x	46.0	68.1	50.3	41.6	65.1	44.9
Agent-Swin-T	276G	3x	47.3	69.5	51.9	42.7	66.4	46.2
Swin-S	358G	1x	45.7	67.9	50.4	41.1	64.9	44.2
Agent-Swin-S	364G	1x	47.2	69.6	52.3	42.7	66.6	45.8
Swin-S	358G	3x	48.5	70.2	53.5	43.3	67.3	46.6
Agent-Swin-S	364G	3x	48.9	70.9	53.6	43.8	67.9	47.3
(b) Cascade Mask R-CNN Object Detection on COCO								
Method	FLOPs	Sch.	AP ^b	AP ₅₀ ^b	AP ₇₅ ^b	AP ^m	AP ₅₀ ^m	AP ₇₅ ^m
Swin-T	745G	1x	48.1	67.1	52.2	41.7	64.4	45.0
Agent-Swin-T	755G	1x	49.2	68.6	53.2	42.7	65.6	45.9
Swin-T	745G	3x	50.4	69.2	54.7	43.7	66.6	47.3
Agent-Swin-T	755G	3x	51.4	70.2	55.9	44.5	67.6	48.4
Swin-S	837G	3x	51.9	70.7	56.3	45.0	68.2	48.8
Agent-Swin-S	843G	3x	52.6	71.3	57.1	45.5	68.9	49.2
Swin-B	981G	3x	51.9	70.5	56.4	45.0	68.1	48.9
Agent-Swin-B	990G	3x	52.6	71.1	57.1	45.3	68.6	49.2

Table 15: Results on COCO object detection with RetinaNet [26]. The FLOPs are computed over backbone, FPN, and detection head with an input resolution of 1280×800.

RetinaNet Object Detection on COCO (Sch. 1x)							
Method	FLOPs	AP	AP ₅₀	AP ₇₅	AP _s	AP _m	AP _l
PVT-T	221G	36.7	56.9	38.9	22.6	38.8	50.0
Agent-PVT-T	211G	40.3	61.2	42.9	25.5	43.4	54.3
PVT-S	286G	38.7	59.3	40.8	21.2	41.6	54.4
Agent-PVT-S	274G	44.1	65.3	47.3	29.2	47.5	59.8
PVT-M	373G	41.9	63.1	44.3	25.0	44.9	57.6
Agent-PVT-M	382G	45.8	66.9	49.1	28.8	49.2	61.7
PVT-L	475G	42.6	63.7	45.4	25.8	46.0	58.4
Agent-PVT-L	492G	46.8	68.2	50.7	30.9	50.8	62.9

Table 16: Results of semantic segmentation. The FLOPs are computed over encoders and decoders with an input image at the resolution of 512×2048 . S-FPN is short for SemanticFPN [23] model.

Semantic Segmentation on ADE20K					
Backbone	Method	FLOPs	#Params	mIoU	mAcc
PVT-T	S-FPN	158G	17M	36.57	46.72
Agent-PVT-T	S-FPN	147G	15M	40.18	51.76
PVT-S	S-FPN	225G	28M	41.95	53.02
Agent-PVT-S	S-FPN	211G	24M	44.18	56.17
PVT-M	S-FPN	315G	48M	42.91	53.80
Agent-PVT-M	S-FPN	321G	40M	44.30	56.42
PVT-L	S-FPN	420G	65M	43.49	54.62
Agent-PVT-L	S-FPN	434G	52M	46.52	58.50
Swin-T	UperNet	945G	60M	44.51	55.61
Agent-Swin-T	UperNet	954G	61M	46.68	58.53
Swin-S	UperNet	1038G	81M	47.64	58.78
Agent-Swin-S	UperNet	1043G	81M	48.08	59.78
Swin-B	UperNet	1188G	121M	48.13	59.13
Agent-Swin-B	UperNet	1196G	121M	48.73	60.01

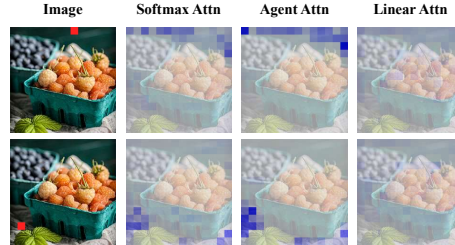


Fig. 10: Visualization of Softmax attention, linear attention and our agent attention. Feature corresponding to the red block is used as query.

Softmax attention with our agent attention in various models results in consistent performance improvements. We further provide additional runtime comparison with other linear attention methods in Fig. 9. Agent attention achieves superior results at a comparable speed compared to other linear attention methods.

Additional downstream experiments. We provide additional experiment results on object detection and semantic segmentation in Tab.15, Tab.14 and Tab.16. For object detection, results on RetinaNet [26], Mask R-CNN [20] and Cascade Mask R-CNN [4] frameworks are presented, while for semantic segmentation, we show results on SemanticFPN [23] and UperNet [51]. It can be observed that our models achieve consistent improvements over their baseline counterparts across various settings.

Ablation on the type of agent tokens. Agent tokens can be acquired through various methods, such as setting as a set of learnable parameters or extracting

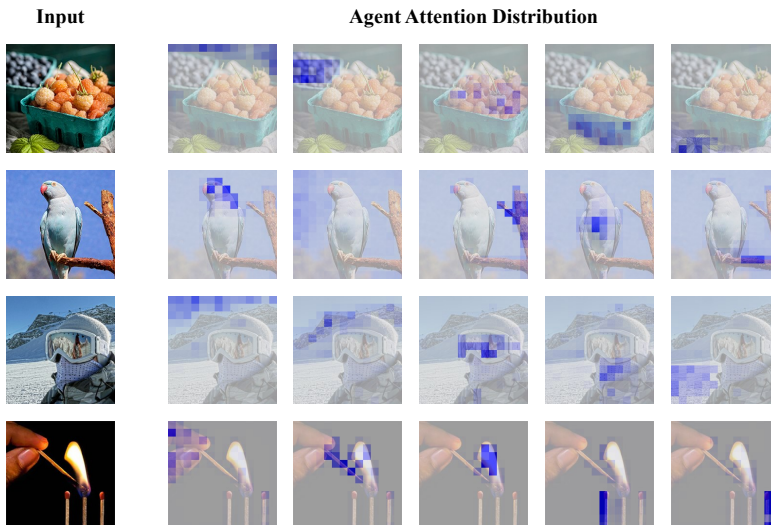


Fig. 11: The distribution of agent attention weights from Agent-Deit-T.

from input features through pooling, depthwise convolution, etc. As shown in Tab. 17, dynamic agents outperform static ones due to their input-dependent nature, allowing for a more accurate representation of current queries. Pooling is a simple yet effective way to acquire agent tokens dynamically.

Table 17: Ablation on different designs of agent tokens.

Designs	Type	FLOPs	#Param	Acc.	Diff.
Learnable Params	Static	4.5G	29M	82.2	-0.4
Pooling	Dynamic	4.5G	29M	82.6	Ours
DWC	Dynamic	4.5G	29M	82.6	+0.0
Deformed Points [50]	Dynamic	4.5G	29M	82.7	+0.1
Token Merging [1]	Dynamic	4.6G	29M	82.6	+0.0

Agent attention at different stages. We conduct ablation study on replacing Softmax attention with our agent attention at different stages. As depicted in Tab. 18, substituting the first three stages results in a performance gain of 1.3, while replacing the final stage marginally decreases overall accuracy. We attribute this outcome to the larger resolutions in the first three stages, which are more conducive to agent attention module with a global receptive field.

F. Agent Attention Visualization

To better understand the effectiveness of agent attention, we provide the visualization of Softmax attention, linear attention and agent attention in Fig. 10. It

Table 18: Applying agent attention module on different stages of the Swin-T structure.

Stages w/ Agent Attn				FLOPS	#Param	Acc.	Diff.
Stage1	Stage2	Stage3	Stage4				
✓				4.5G	29M	81.7	-0.9
✓	✓			4.5G	29M	81.8	-0.8
✓		✓		4.5G	29M	82.6	Ours
✓	✓	✓	✓	4.5G	29M	82.5	-0.1
Swin-T				4.5G	29M	81.3	-1.3

shows that our agent attention produces attention distributions similar to Softmax attention, while linear attention does not generate reasonable distributions. This indicates that agent attention integrates Softmax attention’s expressiveness with linear complexity, resulting in its superiority.

We visualize more agent attention distributions in Fig. 11. It can be seen that various agent tokens focus on distinct regions. For instance, in the second row, agent tokens focus on head, sky, body, and branch, while the third row contains agent tokens focusing on sky, mountain, glasses, mask, and ground. This diversity ensures that different queries can focus on their areas of interest during the agent broadcast process.

G. Model Architectures

Table 19: Architectures of Agent-DeiT models.

stage	output	Agent-DeiT-T		Agent-DeiT-S		Agent-DeiT-B	
		Agent	DeiT Block	Agent	DeiT Block	Agent	DeiT Block
res1	14×14	$\begin{bmatrix} \text{win } 14 \times 14 \\ \text{dim } 192 \\ \text{head } 3 \\ \text{agent } 49 \end{bmatrix} \times 12$	None	$\begin{bmatrix} \text{win } 14 \times 14 \\ \text{dim } 384 \\ \text{head } 6 \\ \text{agent } 49 \end{bmatrix} \times 12$	None	$\begin{bmatrix} \text{win } 14 \times 14 \\ \text{dim } 768 \\ \text{head } 12 \\ \text{agent } 81 \end{bmatrix} \times 4$	$\begin{bmatrix} \text{win } 14 \times 14 \\ \text{dim } 768 \\ \text{head } 12 \end{bmatrix} \times 8$

We present the architectures of four Transformer models used in the main paper, including Agent-DeiT, Agent-PVT, Agent-Swin and Agent-CSwin in Tab.19-23. Considering the advantage of enlarged receptive field, we mainly replace Softmax attention blocks with our agent attention module at early stages of vision Transformer models.

Table 20: Architectures of Agent-PVT models (Part1).

stage	output	Agent-PVT-T		Agent-PVT-S	
		Agent	PVT Block	Agent	PVT Block
res1	56×56	Conv4 $\times$ 4, stride=4, 64, LN			
		$\begin{bmatrix} \text{win } 56 \times 56 \\ \text{dim } 64 \\ \text{head } 1 \\ \text{agent } 9 \end{bmatrix} \times 2$	None	$\begin{bmatrix} \text{win } 56 \times 56 \\ \text{dim } 64 \\ \text{head } 1 \\ \text{agent } 9 \end{bmatrix} \times 3$	None
res2	28×28	Conv2 $\times$ 2, stride=2, 128, LN			
		$\begin{bmatrix} \text{win } 28 \times 28 \\ \text{dim } 128 \\ \text{head } 2 \\ \text{agent } 16 \end{bmatrix} \times 2$	None	$\begin{bmatrix} \text{win } 28 \times 28 \\ \text{dim } 128 \\ \text{head } 2 \\ \text{agent } 16 \end{bmatrix} \times 3$	None
res3	14×14	Conv2 $\times$ 2, stride=2, 320, LN			
		$\begin{bmatrix} \text{win } 14 \times 14 \\ \text{dim } 320 \\ \text{head } 5 \\ \text{agent } 49 \end{bmatrix} \times 2$	None	$\begin{bmatrix} \text{win } 14 \times 14 \\ \text{dim } 320 \\ \text{head } 5 \\ \text{agent } 49 \end{bmatrix} \times 6$	None
res4	7×7	Conv2 $\times$ 2, stride=2, 512, LN			
		$\begin{bmatrix} \text{win } 7 \times 7 \\ \text{dim } 512 \\ \text{head } 8 \\ \text{agent } 49 \end{bmatrix} \times 2$	None	$\begin{bmatrix} \text{win } 7 \times 7 \\ \text{dim } 512 \\ \text{head } 8 \\ \text{agent } 49 \end{bmatrix} \times 3$	None

Table 21: Architectures of Agent-PVT models (Part2).

stage	output	Agent-PVT-M		Agent-PVT-L	
		Agent	PVT Block	Agent	PVT Block
res1	56×56	Conv4 $\times$ 4, stride=4, 64, LN			
		$\begin{bmatrix} \text{win } 56 \times 56 \\ \text{dim } 64 \\ \text{head } 1 \\ \text{agent } 9 \end{bmatrix} \times 3$	None	$\begin{bmatrix} \text{win } 56 \times 56 \\ \text{dim } 64 \\ \text{head } 1 \\ \text{agent } 9 \end{bmatrix} \times 3$	None
res2	28×28	Conv2 $\times$ 2, stride=2, 128, LN			
		$\begin{bmatrix} \text{win } 28 \times 28 \\ \text{dim } 128 \\ \text{head } 2 \\ \text{agent } 16 \end{bmatrix} \times 3$	None	$\begin{bmatrix} \text{win } 28 \times 28 \\ \text{dim } 128 \\ \text{head } 2 \\ \text{agent } 16 \end{bmatrix} \times 8$	None
res3	14×14	Conv2 $\times$ 2, stride=2, 320, LN			
		$\begin{bmatrix} \text{win } 14 \times 14 \\ \text{dim } 320 \\ \text{head } 5 \\ \text{agent } 49 \end{bmatrix} \times 18$	None	$\begin{bmatrix} \text{win } 14 \times 14 \\ \text{dim } 320 \\ \text{head } 5 \\ \text{agent } 49 \end{bmatrix} \times 27$	None
res4	7×7	Conv2 $\times$ 2, stride=2, 512, LN			
		$\begin{bmatrix} \text{win } 7 \times 7 \\ \text{dim } 512 \\ \text{head } 8 \\ \text{agent } 49 \end{bmatrix} \times 3$	None	$\begin{bmatrix} \text{win } 7 \times 7 \\ \text{dim } 512 \\ \text{head } 8 \\ \text{agent } 49 \end{bmatrix} \times 3$	None

Table 22: Architectures of Agent-Swin models.

stage	output	Agent-Swin-T		Agent-Swin-S		Agent-Swin-B	
		Agent	Swin Block	Agent	Swin Block	Agent	Swin Block
res1	56×56	concat $4 \times 4, 96, \text{LN}$		concat $4 \times 4, 96, \text{LN}$		concat $4 \times 4, 128, \text{LN}$	
		$\begin{bmatrix} \text{win } 56 \times 56 \\ \text{dim } 96 \\ \text{head } 3 \\ \text{agent } 9 \end{bmatrix} \times 2$	None	$\begin{bmatrix} \text{win } 56 \times 56 \\ \text{dim } 96 \\ \text{head } 3 \\ \text{agent } 9 \end{bmatrix} \times 2$	None	$\begin{bmatrix} \text{win } 56 \times 56 \\ \text{dim } 128 \\ \text{head } 3 \\ \text{agent } 9 \end{bmatrix} \times 2$	None
res2	28×28	concat $2 \times 2, 192, \text{LN}$		concat $2 \times 2, 192, \text{LN}$		concat $2 \times 2, 256, \text{LN}$	
		$\begin{bmatrix} \text{win } 28 \times 28 \\ \text{dim } 192 \\ \text{head } 6 \\ \text{agent } 16 \end{bmatrix} \times 2$	None	$\begin{bmatrix} \text{win } 28 \times 28 \\ \text{dim } 192 \\ \text{head } 6 \\ \text{agent } 16 \end{bmatrix} \times 2$	None	$\begin{bmatrix} \text{win } 28 \times 28 \\ \text{dim } 256 \\ \text{head } 6 \\ \text{agent } 16 \end{bmatrix} \times 2$	None
res3	14×14	concat $2 \times 2, 384, \text{LN}$		concat $2 \times 2, 384, \text{LN}$		concat $2 \times 2, 512, \text{LN}$	
		None	$\begin{bmatrix} \text{win } 7 \times 7 \\ \text{dim } 384 \\ \text{head } 12 \end{bmatrix} \times 6$	None	$\begin{bmatrix} \text{win } 7 \times 7 \\ \text{dim } 384 \\ \text{head } 12 \end{bmatrix} \times 18$	$\begin{bmatrix} \text{win } 14 \times 14 \\ \text{dim } 512 \\ \text{head } 12 \\ \text{agent } 49 \end{bmatrix} \times 2$	$\begin{bmatrix} \text{win } 7 \times 7 \\ \text{dim } 512 \\ \text{head } 12 \end{bmatrix} \times 16$
res4	7×7	concat $2 \times 2, 768, \text{LN}$		concat $2 \times 2, 768, \text{LN}$		concat $2 \times 2, 1024, \text{LN}$	
		None	$\begin{bmatrix} \text{win } 7 \times 7 \\ \text{dim } 768 \\ \text{head } 24 \end{bmatrix} \times 2$	None	$\begin{bmatrix} \text{win } 7 \times 7 \\ \text{dim } 768 \\ \text{head } 24 \end{bmatrix} \times 2$	None	$\begin{bmatrix} \text{win } 7 \times 7 \\ \text{dim } 1024 \\ \text{head } 24 \end{bmatrix} \times 2$

Table 23: Architectures of Agent-CSwin models.

stage	output	Agent-CSwin-T		Agent-CSwin-S		Agent-CSwin-B	
		Agent	CSwin Block	Agent	CSwin Block	Agent	CSwin Block
res1	56×56	Conv7 $\times$ 7, stride=4, 64, LN				Conv7 $\times$ 7, stride=4, 96, LN	
		$\begin{bmatrix} \text{win } 56 \times 56 \\ \text{dim } 64 \\ \text{head } 2 \\ \text{agent } 9 \end{bmatrix} \times 2$	None	$\begin{bmatrix} \text{win } 56 \times 56 \\ \text{dim } 64 \\ \text{head } 2 \\ \text{agent } 9 \end{bmatrix} \times 3$	None	$\begin{bmatrix} \text{win } 56 \times 56 \\ \text{dim } 96 \\ \text{head } 4 \\ \text{agent } 9 \end{bmatrix} \times 3$	None
res2	28×28	Conv7 $\times$ 7, stride=2, 128, LN				Conv7 $\times$ 7, stride=2, 192, LN	
		$\begin{bmatrix} \text{win } 28 \times 28 \\ \text{dim } 128 \\ \text{head } 4 \\ \text{agent } 16 \end{bmatrix} \times 4$	None	$\begin{bmatrix} \text{win } 28 \times 28 \\ \text{dim } 128 \\ \text{head } 4 \\ \text{agent } 16 \end{bmatrix} \times 6$	None	$\begin{bmatrix} \text{win } 28 \times 28 \\ \text{dim } 192 \\ \text{head } 8 \\ \text{agent } 16 \end{bmatrix} \times 6$	None
res3	14×14	Conv7 $\times$ 7, stride=2, 256, LN				Conv7 $\times$ 7, stride=2, 384, LN	
		None	$\begin{bmatrix} \text{win } 7 \times 14 \\ \text{dim } 256 \\ \text{head } 8 \end{bmatrix} \times 18$	None	$\begin{bmatrix} \text{win } 7 \times 14 \\ \text{dim } 256 \\ \text{head } 8 \end{bmatrix} \times 29$	None	$\begin{bmatrix} \text{win } 7 \times 14 \\ \text{dim } 384 \\ \text{head } 16 \end{bmatrix} \times 29$
res4	7×7	Conv7 $\times$ 7, stride=2, 512, LN				Conv7 $\times$ 7, stride=2, 768, LN	
		None	$\begin{bmatrix} \text{win } 7 \times 7 \\ \text{dim } 512 \\ \text{head } 16 \end{bmatrix} \times 1$	None	$\begin{bmatrix} \text{win } 7 \times 7 \\ \text{dim } 512 \\ \text{head } 16 \end{bmatrix} \times 2$	None	$\begin{bmatrix} \text{win } 7 \times 7 \\ \text{dim } 768 \\ \text{head } 32 \end{bmatrix} \times 2$